

CCNA 200-301 V2.0

22 NAT and PAT on IOS XE

Part IV — Network Services and Security

EXAM TOPICS

4.3

LECTURER

Dr. Mustafa Hameed

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

What you will be able to do

- **Explain** the four NAT address terms and identify each one in a translation table.
- **Configure** static NAT, dynamic NAT and PAT on an IOS XE router, including port forwarding.
- **Interpret** `show ip nat translations` and `show ip nat statistics`.
- **Troubleshoot** the four faults that account for almost every broken NAT: missing interface markers, a wrong ACL, an exhausted pool, and an order-of-operations misunderstanding.

Before we start

Chapter 4 for private address ranges. Chapter 15 — NAT does not replace routing, and a router that cannot route to the internet will not be fixed by NAT. Chapter 6 for port numbers, which is what PAT actually manipulates.

Vocabulary for this chapter

inside local

inside global

outside global

outside local

static NAT

dynamic NAT

PAT

overload

translation table

port forwarding

Each is defined where it first matters — not here.

IOS XE is named in the topic

This is what the blueprint actually asks for

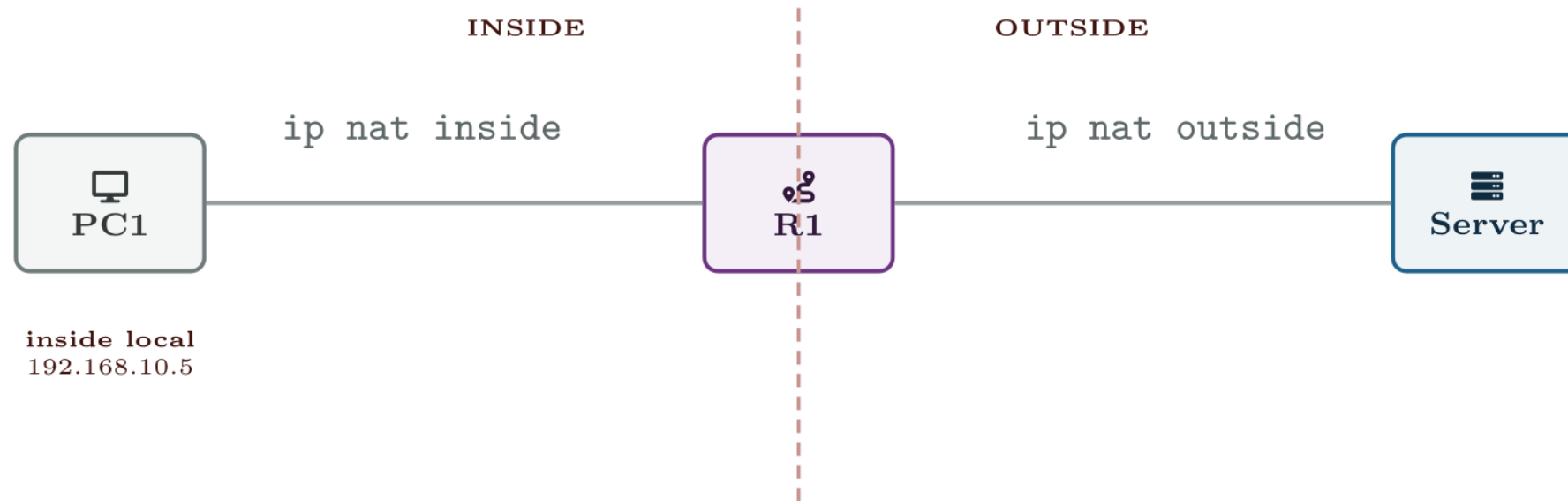
Topic **4.3** is *configure NAT/PAT on IOS XE routers*. The NAT configuration syntax is the same on IOS and IOS XE, so nothing here is wasted on either platform — but the interface names differ (`GigabitEthernet0/0/0` on XE against `GigabitEthernet0/0` on classic IOS), and this chapter uses XE names throughout so that they look normal to you in the exam.

NAT is not a security feature

It is frequently described as one, and it is not. It happens to make unsolicited inbound connections difficult, because there is no translation entry to match them against — but that is a side effect of an address-conservation mechanism, not a policy. Anything you actually want blocked should be blocked by an ACL (Chapter 24) or a firewall, which can be audited and which says what it means.

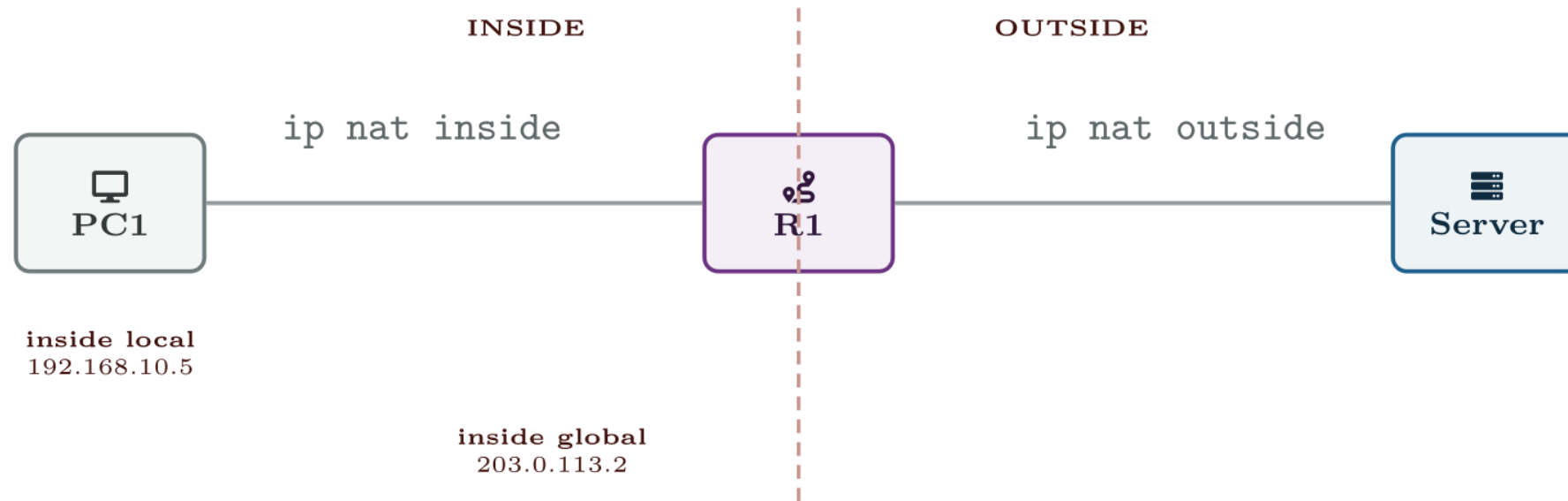
The exam has been known to ask this directly. NAT conserves addresses.

The four terms



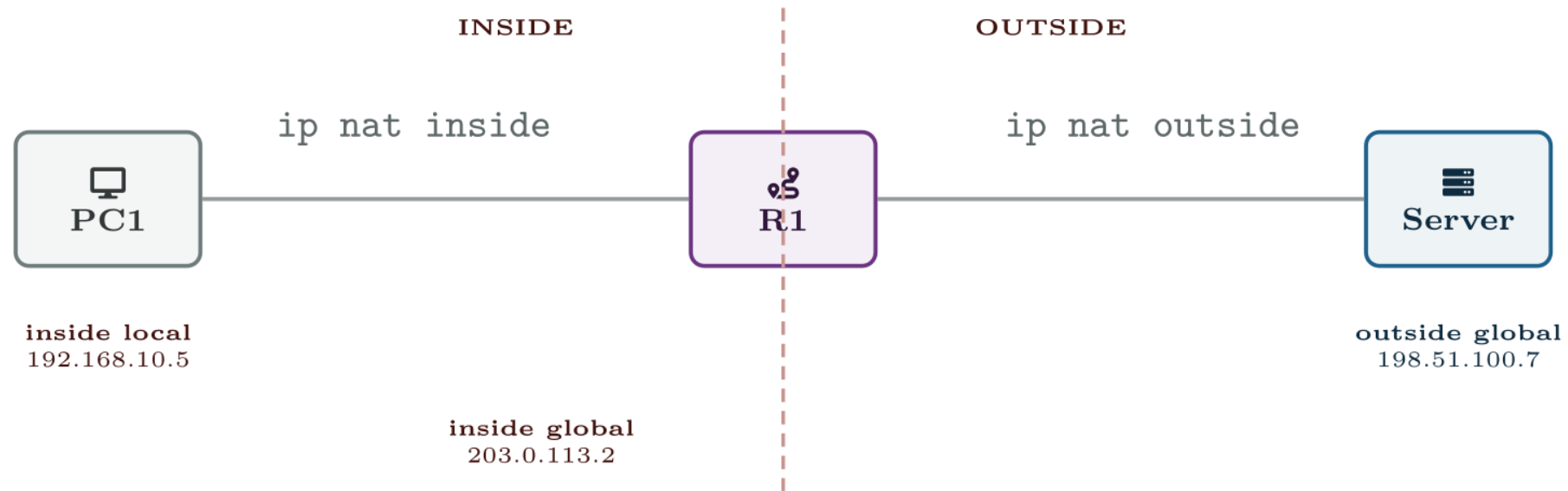
Inside = it is our host. *Local* = the address as the inside sees it. So: the real address on PC1.

The four terms



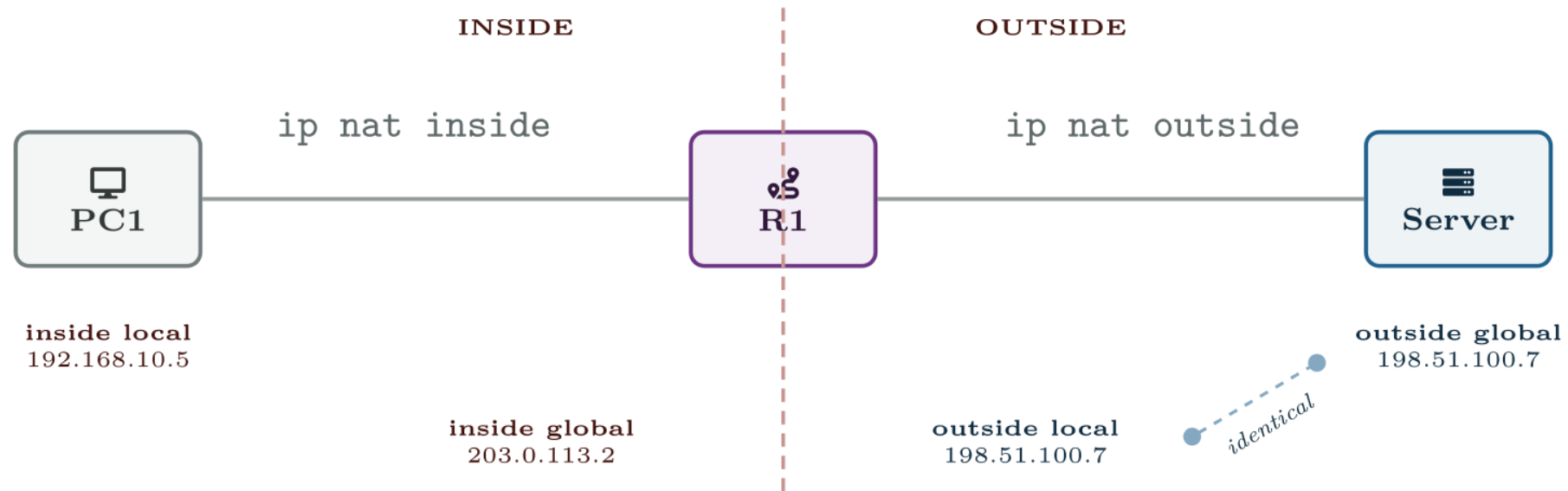
Same host, other viewpoint. *Global* = the address the outside world sees — what R1 translated it to.

The four terms



Now the other host. *Outside* = theirs; *global* = as their side sees it. The server's real address.

The four terms



And the fourth is the same address again — nothing translates the server, so outside local and outside global match. Two hosts, four names.

Why outside local usually equals outside global

Because plain NAT only rewrites the *inside* host's address. The external server's address is untouched in both directions, so both names describe the same number. The two differ only when you translate the outside side as well (`ip nat outside source`), which is rare and outside this exam.

That is good news: for CCNA purposes there are effectively *three* addresses to track, and the one that changes is the inside host's.

Static NAT for an internal web server

IOS · configuration mode

```
! The mapping itself: inside local first, inside global second.
ip nat inside source static 192.168.10.50 203.0.113.50
!
! Without these two lines nothing is translated. This is the most
! commonly forgotten pair of commands in the whole topic.
interface GigabitEthernet0/0/1
  description LAN
  ip address 192.168.10.1 255.255.255.0
  ip nat inside
!
interface GigabitEthernet0/0/0
  description WAN to ISP
  ip address 203.0.113.2 255.255.255.252
  ip nat outside
```

Dynamic NAT from a small pool

IOS · configuration mode

! 1. Which internal addresses are allowed to be translated.

```
access-list 1 permit 192.168.10.0 0.0.0.255
```

!

! 2. The public addresses available to lend.

```
ip nat pool BRANCH-POOL 203.0.113.10 203.0.113.20 netmask 255.255.255.0
```

!

! 3. Tie them together.

```
ip nat inside source list 1 pool BRANCH-POOL
```

Dynamic NAT fails silently when the pool runs out

Eleven addresses in that pool means the twelfth simultaneous host gets no translation and its packets are dropped. There is no message to the user and no log entry unless you asked for one. The symptom is “the internet works in the morning and stops after lunch”, and the evidence is one line of `show ip nat statistics`.

This is exactly why PAT exists, and why almost nobody deploys plain dynamic NAT.

PAT onto the WAN interface address

IOS · configuration mode

```
access-list 1 permit 192.168.10.0 0.0.0.255
!  
! "interface" instead of a pool: use whatever address the WAN  
! interface currently has. This survives a DHCP-assigned WAN address  
! changing, which a pool would not.  
ip nat inside source list 1 interface GigabitEthernet0/0/0 overload  
!  
interface GigabitEthernet0/0/1  
    ip nat inside  
!  
interface GigabitEthernet0/0/0  
    ip nat outside
```

The word overload is the whole difference

This is what the blueprint actually asks for

`ip nat inside source list 1 pool BRANCH-POOL` is dynamic NAT. `ip nat inside source list 1 pool BRANCH-POOL overload` is PAT using a pool. `ip nat inside source list 1 interface Gi0/0/0 overload` is PAT using the interface address.

One keyword changes the behaviour from “runs out” to “does not run out”. Exam questions have been built on exactly that keyword being present or absent.

Publishing two internal services on one public address

IOS · configuration mode

```
ip nat inside source static tcp 192.168.10.50 80 203.0.113.2 80
ip nat inside source static tcp 192.168.10.60 22 203.0.113.2 2222
!
```

! The second line also changes the port: the outside world connects to 2222 and the internal host still listens on 22.

show ip nat translations

R1#

Pro	Inside global	Inside local	Outside local	Outside global
tcp	203.0.113.2:1055	192.168.10.5:1055	198.51.100.7:80	198.51.100.7:80
tcp	203.0.113.2:1056	192.168.10.6:1055	198.51.100.7:80	198.51.100.7:80
tcp	203.0.113.2:80	192.168.10.50:80	---	---

show ip nat statistics

R1#

```
Total active translations: 3 (1 static, 2 dynamic; 2 extended)
Peak translations: 412, occurred 00:31:09 ago
Outside interfaces:
  GigabitEthernet0/0/0
Inside interfaces:
  GigabitEthernet0/0/1
Hits: 20481   Misses: 0
Expired translations: 1173
Dynamic mappings:
-- Inside Source
[Id: 1] access-list 1 interface GigabitEthernet0/0/0 refcount 2
```

The three lines to read first when NAT is broken

- **Inside interfaces / Outside interfaces.** If either list is empty, stop. Nothing else matters until both are populated.
- **Misses.** A non-zero and rising miss count on a dynamic mapping means packets arrived wanting translation and did not get one — an exhausted pool, or an ACL that did not match.
- **Dynamic mappings.** Confirms which ACL and which pool or interface are actually in force, which is not always the one you think you configured.

The consequence you will be asked about

An inbound ACL on the *outside* interface is evaluated **before** the packet is un-translated. So it sees the *inside global* address — **203.0.113.50** — and not the private one. An ACL written against **192.168.10.50** on that interface will never match anything.

An inbound ACL on the *inside* interface is evaluated before translation too, so it sees the private address. Same rule, opposite result: **ACLs always see the address that is on the wire at the interface they are attached to.** If you can hold on to that sentence you never need to memorise the table.

Common pitfalls

- **No `ip nat inside` / `ip nat outside`.** The single most common NAT fault. The translation rule is present, the interfaces are not marked, and nothing happens.
- **Marking the wrong interface as inside.** On a router with several LANs, only the marked ones are translated.
- **A `permit ip any any` ACL used to select NAT traffic.** It will translate traffic destined for other internal subnets too, breaking internal routing in ways that look like a routing fault.
- **Forgetting `overload`.** Works for the first few users and then does not.

Common pitfalls (continued)

- **Expecting to ping the inside global address from inside.** It usually does not work, and it does not mean NAT is broken. Test from outside.
- **Editing a rule without clearing translations.** Existing entries persist. `clear ip nat translation *` after every change, or you are testing the old configuration.
- **Using an ACL number twice.** The NAT selection ACL and a filtering ACL with the same number are the same ACL.

NAT is configured, the translation table is empty, and internal hosts cannot reach the internet.

R1 has a default route, can ping its ISP next hop, and can ping **8.8.8.8** from its own CLI. Internal hosts can ping **R1**'s LAN interface but nothing beyond it. `show ip nat translations` returns nothing at all.

NAT is configured, the translation table is empty, and internal hosts cannot reach the internet.

What would you check first — and what do you expect to see?

show run | include ip nat

R1#

```
ip nat inside source list 1 interface GigabitEthernet0/0/0 overload  
ip nat inside
```

show ip nat statistics

R1#

```
Total active translations: 0 (0 static, 0 dynamic; 0 extended)
Outside interfaces:
Inside interfaces:
  GigabitEthernet0/0/1
Hits: 0   Misses: 0
```

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. `Outside interfaces:` is empty. The WAN interface was never marked `ip nat outside`. NAT needs a boundary, and a boundary needs two sides — with only one side declared, the router has no rule to apply and does not attempt translation at all.

`Hits: 0` and `Misses: 0` together are the confirming detail, and they are worth separating. Misses would mean packets arrived wanting translation and could not get one — a pool or ACL problem. Both counters at zero means no packet was ever *considered* for translation. That distinction points at the interface markers rather than at the rule.

Why it is doing that

The router itself reaching **8.8.8.8** is the red herring. Traffic the router originates is sourced from its own public interface and needs no translation, so it proves the WAN link and the default route are fine and says nothing about NAT.

Declaring the outside boundary

IOS · configuration mode

```
interface GigabitEthernet0/0/0  
  ip nat outside
```

And the lesson

Fix.

Then `clear ip nat translation *` and re-test from a host. Confirm entries appear and that `Hits` is climbing.

PAT for a branch, then a published server

Platform note. Packet Tracer supports this lab, but the topic names IOS XE and the interface naming differs. If you use Packet Tracer, translate the interface names as you go and note where they differ.

Topology. R1 as the branch router with 192.168.10.0/24 inside and 203.0.113.0/30 to ISP; ISP with a loopback of 198.51.100.7 standing in for the internet; three internal hosts and one internal server at 192.168.10.50.

1. Build and verify routing first, with no NAT. Confirm R1 can ping 198.51.100.7 and that internal hosts cannot. Explain precisely why the hosts fail, in terms of the ISP's routing table.
2. Configure PAT using the WAN interface address. Verify from all three hosts and read `show ip nat translations`.
3. Generate connections from two hosts using the same source port (open the same service twice). Find the row where the router had to change the port and record both values.
4. Add a static NAT mapping for the server and confirm from ISP that it is reachable inbound.

PAT for a branch, then a published server (continued)

1. Change the static mapping to port forwarding on port 8080, and confirm the server still listens on 80.
2. **Break 1.** Remove `ip nat outside`. Predict what `show ip nat statistics` will show *before* you look, then look.
3. **Break 2.** Replace PAT with dynamic NAT from a two-address pool, no `overload`. Bring up three simultaneous sessions and observe the third fail. Find the miss counter.
4. **Break 3.** Apply an inbound ACL on the outside interface denying `192.168.10.50`. Confirm it has no effect on inbound traffic to the server, and explain why using the order-of-operations table.

PAT for a branch, then a published server (continued)

- 1. Extension.** Change the NAT selection ACL to `permit ip any any`, add a second internal LAN, and observe what breaks between the two internal subnets.

CHECKPOINT 1 of 5

Give the four NAT terms for a host 10.1.1.9 translated to 198.51.100.4 contacting a server at 203.0.113.80.

Discuss · then advance

Checkpoint 1 of 5

Give the four NAT terms for a host 10.1.1.9 translated to 198.51.100.4 contacting a server at 203.0.113.80.

Inside local 10.1.1.9; inside global 198.51.100.4; outside global 203.0.113.80; outside local 203.0.113.80 — the same, because plain NAT does not translate the outside host's address.

CHECKPOINT 2 of 5

What single keyword turns dynamic NAT into PAT, and what failure does it prevent?

Discuss · then advance

Checkpoint 2 of 5

What single keyword turns dynamic NAT into PAT, and what failure does it prevent?

overload. It translates the port as well as the address, so one public address serves many hosts. Without it, dynamic NAT lends addresses one-to-one and **silently fails** when the pool is exhausted.

CHECKPOINT 3 of 5

`show ip nat statistics` shows Hits: 0 and Misses: 0.
What does that rule out, and what does it point to?

Discuss · then advance

Checkpoint 3 of 5

show ip nat statistics shows Hits: 0 and Misses: 0. What does that rule out, and what does it point to?

It rules out a pool or ACL problem — misses would be non-zero if packets had been considered and refused. Both counters at zero means no packet was ever evaluated for translation, which points at the missing `ip nat inside` or `ip nat outside` interface markers.

CHECKPOINT 4 of 5

An inbound ACL on the outside interface must match a translated inside host. Which address does it have to name, and why?

Discuss · then advance

Checkpoint 4 of 5

An inbound ACL on the outside interface must match a translated inside host. Which address does it have to name, and why?

The **inside global** address, the translated one. An inbound ACL on the outside interface is evaluated *before* the packet is un-translated, so it sees the address that is on the wire at that interface.

CHECKPOINT 5 of 5

Why is NAT not a security control?

Discuss · then advance

Checkpoint 5 of 5

Why is NAT not a security control?

Because it is an address-conservation mechanism. It happens to make unsolicited inbound connections difficult, since no translation entry matches them, but that is a side effect rather than a policy — it cannot be audited, it states no intent, and it is trivially bypassed by anything the inside host initiates. Use an ACL or firewall for policy.

What to take away

- Inside/outside names the host; local/global names the viewpoint. Outside local and outside global are normally the same address.
- Static NAT is a permanent one-to-one map. Dynamic NAT lends from a pool and fails silently when empty. PAT adds `overload` and translates ports too, so it does not run out.
- `ip nat inside` and `ip nat outside` on the interfaces are not optional. Without both, no translation is attempted.
- `show ip nat translations` for what is happening now; `show ip nat statistics` for whether the rule is even being reached.

What to take away (continued)

- ACLs see the address on the wire at the interface they are attached to — translated on the outside, untranslated on the inside.
- `clear ip nat translation *` after every change, or you are testing stale entries.
- NAT conserves addresses. It is not a firewall.

Where we got to

- Inside/outside names the host; local/global names the viewpoint. Outside local and outside global are normally the same address.
- Static NAT is a permanent one-to-one map. Dynamic NAT lends from a pool and fails silently when empty. PAT adds overload and translates ports too, so it does not run out.
- `ip nat inside` and `ip nat outside` on the interfaces are not optional. Without both, no translation is attempted.
- `show ip nat translations` for what is happening now; `show ip nat statistics` for whether the rule is even being reached.

NEXT

Chapter 23 — Device Access, AAA and Secure File Transfer